Đề 06 · Nhập môn Công nghệ nền tảng (RIPT1304) · Tài liệu học nội bộ, không nộp

Hiểu và làm chủ phần việc của Chu Hồng Đức

Sinh viên 4 · SRE: sao lưu, khôi phục, diễn tập sự cố; kiểm thử phân quyền; đánh giá mở rộng; README

Mã sinh viên B24DCCC065 · Lớp RIPT1304-20261-01

Bạn giữ phần "khi hỏng thì sao": sao lưu mỗi ngày, khôi phục thử, đo thời gian gián đoạn và lượng dữ liệu mất, kiểm tra ai được vào đâu, và đề xuất mở rộng lên 10.000, 100.000 sinh viên. Thầy rất hay hỏi phần này vì nó cho thấy nhóm có nghĩ tới sự cố hay không.

Mục lục

1. Phần 1. Kiến thức nền cả nhóm phải nắm
2. Phần 2. Phần việc của bạn
3. Phần 3. Sao lưu và khôi phục từ đầu
4. Phần 4. Kiểm thử 3 và diễn tập sự cố
5. Phần 5. Phân quyền và chặn mạng
6. Phần 6. Sửa và cấu hình trực tiếp
7. Phần 7. Đánh giá mở rộng, chạy lại và số liệu
8. Phần 8. Câu hỏi vấn đáp
9. Phần 9. Việc cần làm trước khi nộp và bảo vệ

> Cách đọc: đọc phần 1 trước (kiến thức nền chung, ai cũng phải nắm), rồi đọc lần lượt các phần riêng của bạn. Chỗ nào có **[mở trên GitHub]** là bấm được, mở đúng tệp và đúng dòng trong repo. Đoạn cấu hình trong tài liệu được trích thẳng từ tệp thật, số dòng bên trái là số dòng trong tệp.

Phần 1. Kiến thức nền cả nhóm phải nắm

1.1 Đề bài yêu cầu gì

Đề 06 của môn yêu cầu: **thiết kế một nền tảng học trực tuyến cho 1.000 sinh viên**. "Thiết kế" ở đây không phải vẽ giao diện, mà là trả lời bằng số các câu hỏi của người làm hệ thống:

- 1.000 sinh viên thì **cùng lúc có bao nhiêu người** dùng? Cần bao nhiêu **băng thông** (đường truyền mạng), bao nhiêu **dung lượng lưu trữ**?
- Hệ thống gồm những phần nào, nối với nhau ra sao (**kiến trúc**)?
- Dựng thử được không, **đo** được không: chậm ở đâu, chịu được bao nhiêu người?
- Khi hỏng (máy chết, mất dữ liệu) thì **khôi phục** thế nào, mất bao lâu, mất bao nhiêu dữ liệu?

Phiếu đề chia thành 4 yêu cầu và 3 bài kiểm thử. Nhóm gọi bằng lời thường như sau:

Phần của đề	Nội dung	Ai lo chính
Yêu cầu 1: thiết kế	Thiết kế web, dữ liệu, video, đăng nhập; phân biệt **tổng tài khoản** với **số người dùng cùng lúc**	Đăng
Yêu cầu 2: tính tài nguyên	Tính băng thông, lưu trữ cho ít nhất 2 tình huống, ghi rõ mọi giả định	Đăng
Yêu cầu 3: triển khai	Dựng 1 web học liệu nhỏ có ít nhất 3 tài nguyên (video, PDF, bài nghe); thử 3 mức tải	Thành (web), Đăng (lab)
Yêu cầu 4: đo và đề xuất	Đo thời gian trả lời, số request mỗi giây, lỗi, CPU/RAM; chỉ ra điểm nghẽn; sao lưu; phân quyền	Vân Anh (đo), Đức (sao lưu, quyền)
Kiểm thử 1	Máy khách mở được trang và tải đúng học liệu	Vân Anh, Thành
Kiểm thử 2	Chạy 3 mức tải, ghi số liệu, điều kiện đo lặp lại được	Vân Anh
Kiểm thử 3	Dừng dịch vụ rồi khôi phục; kiểm tra học liệu và thời gian gián đoạn	Đức

Hồ sơ phải nộp (mục "Hồ sơ nộp chung" của phiếu đề): báo cáo nhóm 12–18 trang; sơ đồ và bảng tài nguyên/IP/quyền; cấu hình, script, mã nguồn; README hướng dẫn dựng lại; bảng kiểm thử có giờ chạy; dữ liệu đo gốc; video demo 5–7 phút; báo cáo cá nhân. Tất cả nằm trong thư mục ho-so-nop.

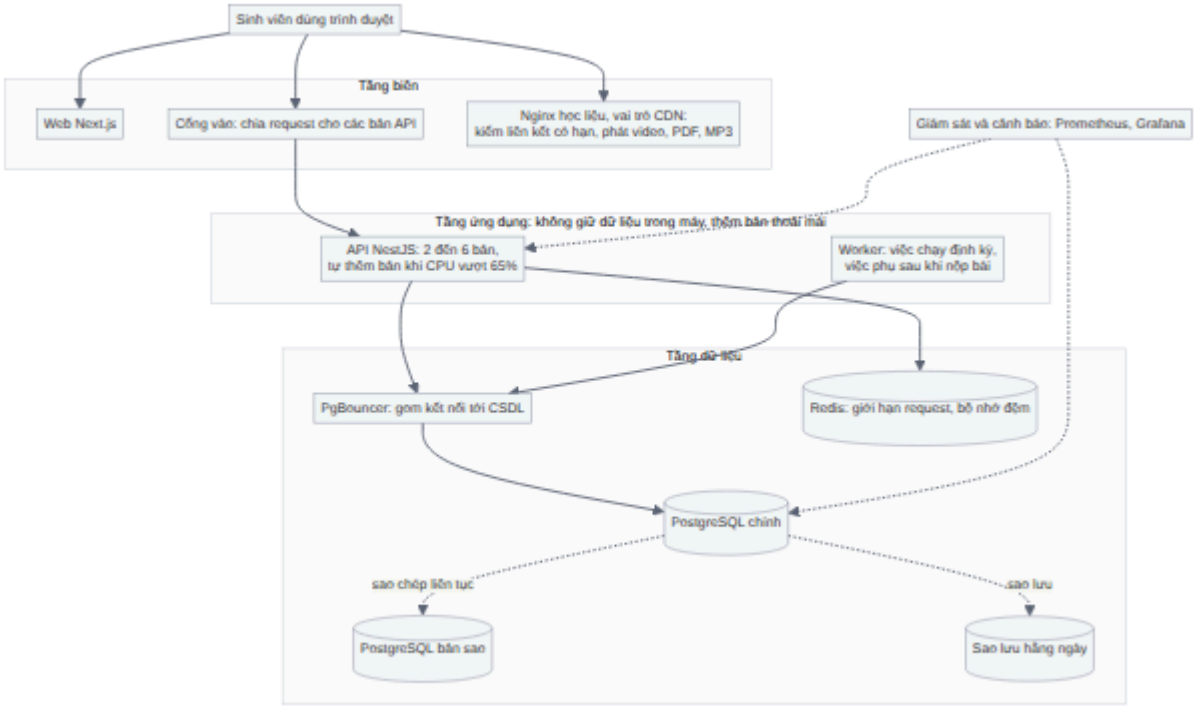
Buổi bảo vệ: mỗi người trình bày phần mình, **mở minh chứng** (tệp kết quả), **làm 1 thao tác hoặc sửa 1 thay đổi nhỏ** ngay trước mặt thầy, rồi **trả lời vấn đáp** (thầy có thể hỏi cả kiến trúc chung). Điểm cá nhân chiếm 50/100: 15 điểm phần kỹ thuật có bằng chứng, 15 điểm giải thích và thao tác trực tiếp, 20 điểm vấn đáp. Tài liệu này giúp bạn làm tốt cả 3 việc đó.

> Hanni chỉ là **hệ thống minh hoạ** (web học tiếng Trung mà Đăng viết từ trước). Thứ được chấm là cách nhóm thiết kế, đo, khôi phục, mở rộng, chứ không phải tính năng học tiếng Trung.

1.2 Hệ thống gồm những phần nào (ví như một quán ăn)

Hãy hình dung hệ thống như một quán ăn đông khách:

Phần trong hệ thống	Ví như	Làm gì
Trình duyệt của sinh viên	Khách hàng	Gửi yêu cầu, nhận kết quả, hiện lên màn hình
Web (Next.js, trang `/hoc-lieu`)	Thực đơn và bàn ăn	Phần giao diện sinh viên nhìn thấy, bấm vào
API (NestJS)	Bếp	Nhận yêu cầu ("cho tôi danh sách học liệu"), xử lý, trả kết quả. Chạy nhiều bản như nhiều đầu bếp
CSDL PostgreSQL	Kho nguyên liệu, sổ sách	Nơi giữ dữ liệu thật: tài khoản, tiến độ học, bài thi. Mất là mất thật
Redis	Bảng ghi chú dán tường	Ghi tạm những thứ cần đọc nhanh, mất cũng dựng lại được (bộ đếm giới hạn, bộ nhớ đệm)
PgBouncer	Người giữ cửa kho	Gom yêu cầu vào kho của nhiều đầu bếp, chỉ cho một số lượng cố định vào cùng lúc để kho không quá tải
Nginx học liệu (vai CDN)	Quầy phát đồ đóng gói sẵn	Phát thẳng video, PDF, MP3 cho khách, không bắt bếp phải làm. Khách phải có "phiếu" (liên kết có hạn)
Worker	Nhân viên hậu cần	Làm việc nền: nhắc học hằng ngày, xử lý việc phụ sau khi nộp bài
Giám sát (Prometheus, Grafana, Alertmanager)	Camera và chuông báo	Đo mọi thứ liên tục, vẽ biểu đồ, kêu khi có sự cố
Sao lưu	Bản photo sổ sách cất két	Mỗi ngày chép CSDL và học liệu ra chỗ khác để hỏng còn lấy lại được



Hình 1. Kiến trúc đề xuất cho 1.000 sinh viên (tệp gốc: ho-so-nop/so-do-va-bang/kien-truc-dich.png)

Ba tầng trong hình: tầng biên (web, cổng vào, Nginx học liệu) là chỗ khách đi vào; tầng ứng dụng (API, worker) là chỗ xử lý; tầng dữ liệu (PostgreSQL, Redis, sao lưu) là chỗ giữ dữ liệu. Ý quan trọng nhất: **tầng ứng dụng không giữ dữ liệu trong máy**, mọi thứ cần nhớ đều nằm ở CSDL hoặc Redis, nên muốn chịu nhiều người hơn chỉ cần **thêm bản API**, không phải sửa code.

1.3 Từ điển thuật ngữ (đọc kỹ, gặp lại ở khắp tài liệu)

Mạng và web

- **Request (yêu cầu)**: một lần trình duyệt hỏi máy chủ, ví dụ "cho tôi danh sách học liệu". Mỗi lần bấm nút có thể sinh ra vài request.
- **Response (trả lời)**: máy chủ đáp lại request, kèm một **mã trả về** (mã HTTP) cho biết kết quả.
- **API**: tập các "cửa" mà máy chủ mở ra để trình duyệt gọi vào, mỗi cửa là một **route** (đường dẫn), ví dụ `GET /api/resources` (lấy danh sách học liệu), `POST /api/resources/:id/link` (xin liên kết tải). `GET` là đọc, `POST` là gửi hoặc tạo mới.
- **Mã trả về HTTP** hay gặp trong dự án:

Mã	Nghĩa	Gặp ở đâu trong dự án
200	Thành công	Gọi API bình thường
401	Chưa đăng nhập	Khách xin liên kết video cần đăng nhập
403	Đã biết là ai nhưng không có quyền, hoặc chữ ký sai	Sửa chữ ký liên kết học liệu; người học gọi chức năng quản trị
404	Không có thứ này (hoặc giấu đi cho an toàn)	Gọi `/api/metrics` từ ngoài
410	Đã từng có nhưng hết hạn	Dùng liên kết học liệu đã quá 10 phút
429	Gửi quá nhiều, chờ chút rồi thử lại	Một tài khoản gọi quá 300 request trong 1 phút
503	Máy chủ đang quá tải hoặc chưa sẵn sàng, "thử lại sau"	Cơ chế từ chối khi quá tải của nhóm; API mất CSDL
5xx khác (500, 502…)	Lỗi máy chủ thật	Trong các lượt đo chính thức: 0 lỗi loại này

- **Băng thông**: lượng dữ liệu đi qua đường mạng mỗi giây, đơn vị **Mbps** (megabit mỗi giây). Chú ý: 8 bit = 1 byte, nên 8 Mbps = 1 MB mỗi giây. Video 540p của nhóm là 2 Mbps, tức mỗi giây xem tốn khoảng 0,25 MB.

Máy chủ

- **CPU, lõi (core)**: bộ xử lý. "0,25 lõi" nghĩa là được dùng một phần tư sức một lõi. Trong Kubernetes viết `250m` (250 phần nghìn lõi).
- **RAM**: bộ nhớ đang làm việc, viết `256Mi` (256 mebibyte, gần bằng 256 MB). Hết RAM thì máy phải dùng **swap** (lấy ổ đĩa làm RAM tạm), chậm hẳn đi; vì vậy lần đo nào máy bị swap thì nhóm bỏ, đo lại.
- **Tiến trình**: một chương trình đang chạy, ví dụ 1 bản API là 1 tiến trình Node.js.
- **Event loop (vòng xử lý của Node.js)**: Node.js xử lý mọi request bằng một vòng lặp duy nhất. Khi CPU quá bận, vòng này bị **trễ** (event loop lag): request phải xếp hàng chờ dù chưa cần tới CSDL.

Nhóm đo độ trễ này để biết API đang quá tải.

Docker và Kubernetes

- **Docker, container, image**: Docker đóng gói một chương trình cùng mọi thứ nó cần vào một **image** (như hộp cơm đóng sẵn). Chạy image lên thành **container** (hộp cơm đang được ăn). Nhờ vậy máy nào có Docker cũng chạy được giống hệt nhau. Mọi công cụ của lab (k6, ffmpeg, kubectl) đều chạy trong container nên máy không phải cài gì thêm.
- **Kubernetes**: phần mềm **quản lý nhiều container** trên nhiều máy: tự bật lại container chết, tự thêm bản khi đông, chia request đều. **k3s** là bản Kubernetes nhẹ, nhóm chạy nó ngay trong Docker để dựng lab trên 1 máy.
- **Cụm (cluster)**: tập các máy Kubernetes quản lý. **Node**: mỗi máy trong cụm. Lab có 2 node: `k3s-server` (172.25.0.2) và `k3s-agent` (172.25.0.3).
- **Pod**: đơn vị nhỏ nhất Kubernetes chạy, thường là 1 container. 1 bản API = 1 pod.
- **Deployment**: bản mô tả "tôi muốn chạy N pod giống nhau từ image X". Kubernetes giữ đúng N pod, chết cái nào tạo lại cái đó.
- **Bản (replica)**: mỗi pod giống nhau trong một Deployment. "API chạy 6 bản" là 6 pod API cùng phục vụ.
- **Service**: một địa chỉ cố định đứng trước nhiều pod, chia request cho các pod đó. Pod chết, đổi địa chỉ, Service vẫn giữ nguyên.
- **Ingress (cổng vào)**: cửa chính từ ngoài vào cụm, xem đường dẫn để đưa request tới đúng Service (`api.hanni.localhost` tới API, `media.hanni.localhost` tới Nginx học liệu). Lab dùng Traefik làm cổng vào, mở ở `127.0.0.1:18080`.
- **Namespace**: ngăn chia trong cụm. Ứng dụng nằm ở `hanni`, giám sát nằm ở `observability`.
- **Manifest (tệp YAML)**: tệp mô tả thứ muốn chạy (Deployment, Service…). Nhóm sửa manifest rồi **apply** (nạp) vào cụm.
- **kubectl**: lệnh điều khiển cụm. Trong lab gọi qua `scripts/kubectl` (đã nối sẵn vào lab). Vài lệnh hay dùng: `scripts/kubectl -n hanni get pods` (xem các pod), `get hpa` (xem tự mở rộng), `logs <tên pod>` (xem nhật ký).
- **HPA (tự mở rộng)**: Horizontal Pod Autoscaler, cứ 15 giây đo CPU trung bình các bản API; vượt ngưỡng (65% mức CPU yêu cầu) thì thêm bản, thấp lâu thì bớt, trong khoảng 2–6 bản.
- **Probe (kiểm tra sức khoẻ)**: Kubernetes định kỳ gọi thử vào pod. **Readiness** hỏi "sẵn sàng nhận khách chưa?", chưa thì tạm không chia request cho pod đó. **Liveness** hỏi "còn sống không?", nhiều lần không trả lời thì khởi động lại pod.

Dữ liệu

- **CSDL (cơ sở dữ liệu) PostgreSQL**: nơi lưu dữ liệu dạng bảng (bảng `User`, `ReviewLog`…), mỗi dòng là một bản ghi.
- **Kết nối CSDL, pool**: mỗi lần API nói chuyện với CSDL phải dùng một **kết nối**. Pool là "hồ" kết nối mở sẵn để dùng lại. CSDL chỉ chịu được một số kết nối nhất định; hết kết nối thì request phải **xếp hàng chờ**, đây là điểm nghẽn đầu tiên nhóm đo được.
- **Bản sao (replica) CSDL**: một CSDL thứ hai liên tục chép theo CSDL chính, dùng để đọc bớt và để dự phòng khi CSDL chính hỏng.
- **WAL (nhật ký thay đổi)**: PostgreSQL ghi mọi thay đổi vào nhật ký trước. Nhờ nhật ký này mà bản sao chép theo được, và có thể khôi phục về **đúng một thời điểm** trước khi xoá nhầm.

- **Redis**: kho dữ liệu cực nhanh nằm trong RAM, dùng cho thứ mất cũng không sao: bộ đếm giới hạn request, bộ nhớ đệm.
- **Bộ nhớ đệm (cache)**: lưu tạm kết quả đã tính để lần sau trả ngay, không phải tính lại. Nhóm lưu tạm "khung lộ trình học" 60 giây.

Đo hiệu năng

- **Tải**: lượng người và request đổ vào hệ thống. Đo bằng **request/giây** hoặc **lượt/giây** (1 lượt mở bài = 5 request đọc).
- **Người dùng cùng lúc (phiên đồng thời)**: số người đang thật sự dùng ở cùng một thời điểm. 1.000 tài khoản không có nghĩa 1.000 người cùng lúc: tự học rải rác chỉ khoảng **63 người**, cả lớp xem video 300, thi thử đồng loạt 600.
- **Định luật Little**: số người cùng lúc = số người bắt đầu mỗi phút × số phút mỗi người ở lại. Ví như quán ăn: mỗi phút có 2,5 khách vào, mỗi khách ngồi 25 phút, thì lúc nào cũng có khoảng 63 khách trong quán.
- **Thời gian trả lời** (độ trễ): từ lúc gửi request tới lúc nhận trả lời, đơn vị **ms** (mili giây, 1.000 ms = 1 giây).
- **p50, p95, p99 (phân vị)**: xếp thời gian trả lời của mọi request từ nhanh tới chậm. **p95 = 40 ms** nghĩa là 95% request nhanh hơn 40 ms, 5% chậm hơn. Nhóm dùng p95 vì **trung bình** bị số đông nhanh che mất những người phải chờ lâu.
- **Thông lượng**: số request trả lời được mỗi giây.
- **Tỉ lệ lỗi**: phần trăm request bị lỗi.
- **SLO (ngưỡng đạt)**: mức chất lượng nhóm tự đặt ra: **95% lượt đọc dưới 300 ms, 95% lượt ghi dưới 500 ms, lỗi dưới 0,5%**. Đo mà vượt ngưỡng là "không đạt".
- **Điểm gãy**: mức tải cao nhất còn đạt SLO; tăng quá đó là chậm vọt. Lab đo được khoảng 40 lượt/giây (khoảng 200 request/giây).
- **Điểm nghẽn (nút thắt)**: bộ phận bị đầy đầu tiên khiến cả hệ thống chậm. Như con đường: chỗ hẹp nhất quyết định xe chạy được bao nhiêu.
- **Từ chối khi quá tải**: khi quá tải, API trả lời ngay mã 503 "thử lại sau" cho một phần lượt đọc, thay vì nhận hết rồi bắt tất cả chờ 5 giây. Người được phục vụ thì nhanh, người bị từ chối thì biết để thử lại.
- **k6**: công cụ **giả lập nhiều người dùng** gửi request đều đặn rồi đo thời gian trả lời. Nhóm viết **kịch bản** k6 bằng JavaScript.
- **Lab**: môi trường đo riêng của nhóm (cụm k3s 2 node trong Docker, dữ liệu giả 1.000 tài khoản), tách hẳn web thật để đo không ảnh hưởng người dùng.
- **GitHub Actions**: dịch vụ của GitHub cho mượn máy ảo chạy lệnh tự động. Laptop của nhóm không đủ RAM nên **số đo chính thức** chạy trên máy ảo GitHub (4 nhân, 16 GB) ngày 09/11/2026; máy ảo dựng lại lab từ đầu theo README rồi mới đo.

Giám sát và sao lưu

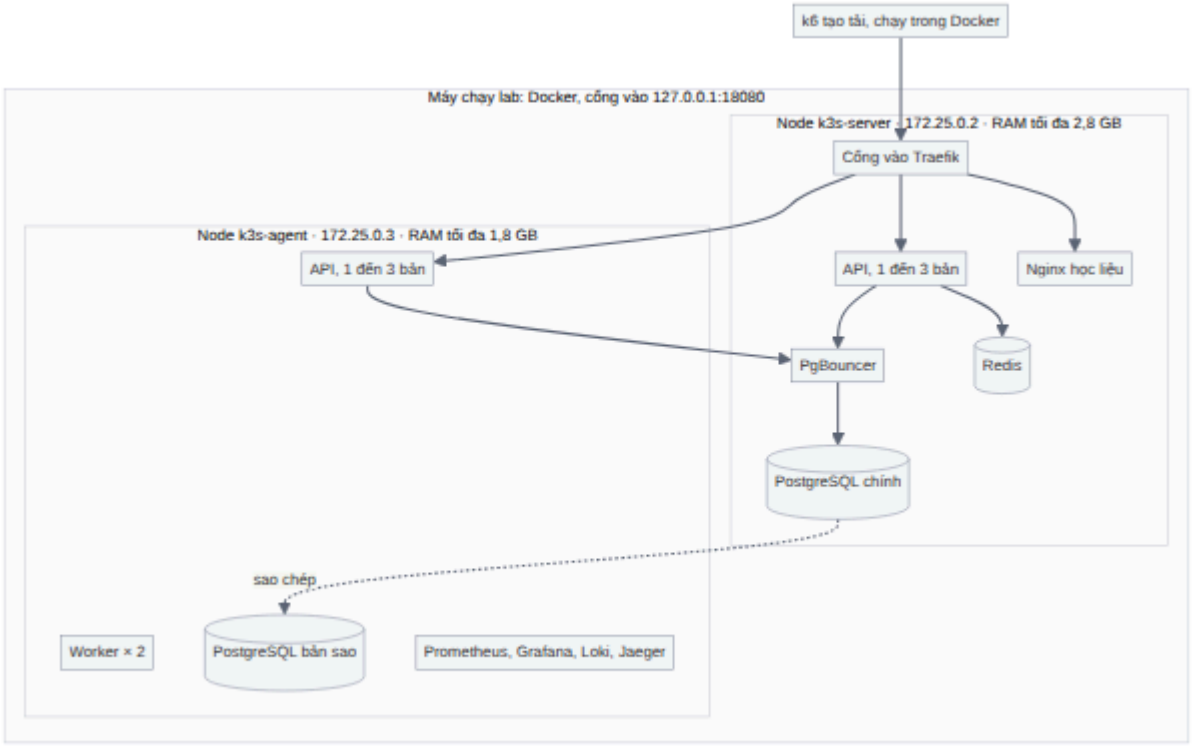
- **Prometheus**: cứ vài giây hỏi số liệu của từng thành phần (API, CSDL, Redis) rồi lưu lại theo thời gian.
- **Grafana**: vẽ số liệu của Prometheus thành biểu đồ. Nhóm có 3 bảng: chất lượng dịch vụ, mở rộng (số bản API, CPU), dữ liệu.

- **Alertmanager**: gửi cảnh báo khi một luật bị vi phạm, ví dụ "từ ngoài gọi API không được quá 30 giây".
- **Blackbox (dò từ ngoài)**: một chương trình đứng ngoài gọi thử API mỗi vài giây như một sinh viên thật, để biết sinh viên có vào được không.
- **Loki, Jaeger**: Loki gom nhật ký (log) của mọi pod; Jaeger lần theo đường đi của một request qua các phần.
- **Sao lưu (backup)**: chép dữ liệu ra một bản riêng để khi hỏng lấy lại được. **Khôi phục (restore)**: đưa bản sao lưu trở lại.
- **RPO**: lượng dữ liệu tối đa có thể mất, tính từ lần sao lưu gần nhất. Sao lưu mỗi ngày thì RPO tối đa 1 ngày.
- **RTO**: thời gian từ lúc hỏng tới lúc dùng lại được. Lab đo được RTO 52 giây khi xoá hẳn CSDL.
- **Mã băm SHA-256**: "dấu vân tay" của tệp, một chuỗi 64 ký tự tính từ nội dung. Sai một byte là chuỗi khác hẳn, nên so mã băm biết tệp tải về hay khôi phục ra có đúng y nguyên không.

Công cụ dựng lab

- **make, Makefile**: `make up` là chạy một nhóm lệnh đã viết sẵn trong tệp `Makefile`. Nhóm có `make up`, `make images`, `make deploy`, `make seed`, `make observability`, `make status`, `make stop`, `make start`, `make down`.
- **YAML**: định dạng các tệp cấu hình. Quy tắc: `khoá: giá trị`, **thụt lề bằng dấu cách** thể hiện cấp (sai thụt lề là hỏng tệp), dòng bắt đầu bằng `#` là ghi chú, chuỗi số nên để trong ngoặc kép như `"600"`.

1.4 Lab của nhóm trông như thế nào



Hình 2. Sơ đồ lab: 2 node, địa chỉ IP và các thành phần (tệp gốc: ho-so-nop/so-do-va-bang/so-do-lab.png)

Thành phần	Địa chỉ	Ghi chú
Node k3s-server	172.25.0.2	RAM tối đa 2,8 GB
Node k3s-agent	172.25.0.3	RAM tối đa 1,8 GB
Cổng vào (ingress)	127.0.0.1:18080	Mọi địa chỉ dưới đây đi qua cổng này
API	http://api.hanni.localhost:18080/api/health	
Grafana	http://grafana.hanni.localhost:18080	Tài khoản `admin`, mật khẩu lấy bằng lệnh ở README
Prometheus	http://prometheus.hanni.localhost:18080	
Alertmanager	http://alertmanager.hanni.localhost:18080	
Web	http://hanni.localhost:3000	Chạy riêng bằng `npx next dev`

1.5 Repo này có gì, phần của mình ở đâu

Đường dẫn	Nội dung
ho-so-nop/bao-cao/	Báo cáo nhóm (.docx, .pdf), 4 báo cáo cá nhân theo mẫu của thầy
ho-so-nop/so-do-va-bang/	2 sơ đồ, bảng tài nguyên/IP/quyền kèm bảng tính tài nguyên
ho-so-nop/cau-hinh-ma-nguon/hanni-server/infra/	**Toàn bộ cấu hình lab**: k8s/ (Kubernetes), k6/ (kịch bản tải), scripts/ (script kiểm thử, sao lưu), observability/ (giám sát), media/ (học liệu)
ho-so-nop/cau-hinh-ma-nguon/hanni-server/src/	Mã API nhóm viết cho đề: học liệu, số liệu đo, giới hạn request, từ chối khi quá tải
ho-so-nop/cau-hinh-ma-nguon/hanni-client/	Trang web học liệu và test tự động
ho-so-nop/data/raw/	Dữ liệu đo gốc, không sửa tay. `gha-09-11/` là lượt đo chính thức
ho-so-nop/evidence/	Minh chứng: ảnh chụp, nhật ký, danh sách mã băm. Danh mục ở `evidence/README.md`
ho-so-nop/bang-kiem-thu.xlsx	Bảng kiểm thử 62 dòng có giờ chạy
ho-so-nop/video-demo.mp4	Video demo 6 phút
Hanni-RIPT1304-Phan-cong.xlsx	Sheet phân công: việc của từng người, tiến độ, tab **Ôn vấn đáp**

Mã nguồn đầy đủ (chạy được) ở 2 repo: hanni-server nhánh feat/scaling và hanni-client nhánh feat/hoc-lieu. Thư mục `cau-hinh-ma-nguon/` chỉ là bản trích các tệp nhóm viết cho đề.

1.6 Dựng lab trên máy của bạn

Cần: Docker (kèm Docker Compose), `make`, `git`, còn trống khoảng 3 GB RAM và 8 GB đĩa. Windows thì dùng WSL 2.

```
git clone -b feat/scaling https://github.com/DANG-PH/hanni-server.git
cd hanni-server/infra
make up              # tạo học liệu mẫu, bật 2 node k3s, sinh mật khẩu ngẫu nhiên
make images          # build image API rồi nạp vào 2 node
make deploy          # chạy PostgreSQL + bản sao, PgBouncer, Redis, API, worker, Nginx học liệu, tự
mở rộng, chặn mạng
make seed            # tạo bảng, nạp từ vựng, tạo 1.000 tài khoản giả
make observability   # chạy Prometheus, Alertmanager, Grafana, Loki, Jaeger
make status          # mọi pod phải ở trạng thái Running
```

Lần đầu mất 10–20 phút vì phải tải image; các lần sau khoảng 4–5 phút. Tạm tắt cho nhẹ máy: `make stop`, bật lại: `make start`, xoá sạch: `make down`. Nếu máy yếu không dựng nổi, nhờ Đăng chạy trên GitHub Actions (đẩy nhánh lên `do-lab` là máy ảo tự dựng và đo).

1.7 Quy tắc chung khi sửa cấu hình

1. Mở tệp cần sửa trong `hanni-server/infra` (đường dẫn ở bảng "Sửa nhanh" của phần riêng bạn), sửa đúng dòng, giữ nguyên thụt lề.
2. Nạp lại vào cụm: `make deploy`.
3. Nếu đã sửa `k8s/app/config.yaml` hoặc `k8s/media-manifest.json` thì chạy thêm `scripts/kubectl -n hanni rollout restart deployment/hanni-api`, vì API chỉ đọc 2 tệp này lúc khởi động.
4. Nếu đã sửa luật cảnh báo trong `observability/` thì chạy `make observability` rồi `scripts/kubectl -n observability rollout restart deployment/prometheus`.
5. **Kiểm tra lại** bằng lệnh hoặc trang ghi ở cột "Kiểm tra" và nói được kết quả thay đổi thế nào.

> Khi thầy bảo "đổi X rồi chạy lại", hãy nói to 3 ý: (1) X nằm ở tệp nào, dòng nào; (2) đổi xong thì hệ thống sẽ thay đổi gì; (3) lệnh nào kiểm tra được điều đó. Làm như vậy là ăn trọn điểm "giải thích và thao tác trực tiếp".

1.8 Lỗi hay gặp khi dựng lab và cách xử lý

Cách tự tìm lỗi trước khi hỏi: (1) `make status` xem pod nào không ở trạng thái `Running`; (2) `scripts/kubectl -n hanni describe pod <tên pod>`, đọc mục **Events** ở cuối (Kubernetes ghi lý do bằng tiếng Anh, ví dụ thiếu RAM, thiếu image); (3) `scripts/kubectl -n hanni logs <tên pod>` xem nhật ký của chính chương trình.

Hiện tượng	Nguyên nhân	Cách xử lý
`permission denied ... docker.sock` khi chạy `make`	Tài khoản chưa có quyền dùng Docker	`sudo usermod -aG docker $USER`, đăng xuất rồi đăng nhập lại (hoặc chạy lệnh bằng `sudo`)
`port is already allocated` khi `make up`	Cổng 18080 đã có chương trình khác dùng	Tắt chương trình đó, hoặc đổi số 18080 ở dòng `127.0.0.1:18080:80` trong `compose.yaml` (mọi địa chỉ đổi theo)
Lần đầu `make up`, `make images` rất lâu	Phải tải image k3s, PostgreSQL, Node.js…	Bình thường, 10–20 phút lần đầu, các lần sau nhanh

Hiện tượng	Nguyên nhân	Cách xử lý
Pod API báo `ErrImageNeverPull`	Chưa chạy `make images`: image API build tại chỗ, manifest đặt `imagePullPolicy: Never` (không tải từ mạng)	`make images` rồi `make deploy`
Pod `Pending` hoặc `OOMKilled`, máy rất chậm	Thiếu RAM: 2 node được dùng tới 2,8 GB + 1,8 GB	Đóng bớt ứng dụng; chỉ thử API thì bỏ `make observability`; máy dưới 8 GB RAM thì nhờ Đăng chạy trên GitHub Actions
`kubectl exec` báo lỗi chứng chỉ sau khi khởi động lại máy	Docker đổi IP của node	`compose.yaml` đã cố định IP; nếu vẫn gặp: `make down` rồi dựng lại
Đăng nhập trên web xong vẫn như khách	Mở `localhost:3000` thay vì `hanni.localhost:3000`, trình duyệt không gửi cookie sang API	Mở đúng `http://hanni.localhost:3000`
k6 báo vài % lỗi ở đầu lần đo, mức cao hơn lại không lỗi	Đo đúng lúc Kubernetes đang thêm bản API	Chờ đủ bản rồi mới đo (`scripts/cho-du-ban.sh`; `chay-kt2.sh` đã tự làm)
Sau khi chạy `failover.sh` lab chạy lạ	Script cố ý tắt CSDL chính và đưa bản sao lên thay, lab không còn như lúc đầu	`make down` rồi dựng lại từ đầu trước khi đo tiếp

Phần 2. Phần việc của bạn

2.1 Phiếu đề giao gì cho bạn

Phiếu đề ghi Sinh viên 4 lo **"Sao lưu, phục hồi và đánh giá mở rộng"**, kèm Kiểm thử 3: dừng dịch vụ rồi khôi phục, kiểm tra học liệu và thời gian gián đoạn.

Vai trò	Bạn làm gì	Phục vụ phần nào của đề
SRE (kỹ sư độ tin cậy)	Sao lưu tự động, khôi phục thử, diễn tập sự cố, đo RPO/RTO	Kiểm thử 3, Yêu cầu 4 (sao lưu)
Kiểm thử phân quyền	9 ca kiểm tra ai được làm gì, chặn mạng trong cụm	Yêu cầu 4 (kiểm soát truy cập)
Đánh giá mở rộng	Đề xuất cho 100 / 1.000 / 10.000 / 100.000 sinh viên theo điểm nghẽn đã đo	Yêu cầu 4
Technical Writer	README dựng lại hệ thống, chạy kiểm thử	Hồ sơ nộp

> "SRE" (Site Reliability Engineering) là người lo cho hệ thống **chạy ổn định**: đo, cảnh báo, khôi phục, tập xử lý sự cố. Không cần viết nhiều code, nhưng phải hiểu hệ thống hỏng kiểu gì và lấy lại thế nào.

2.2 Sản phẩm của bạn trong hồ sơ

Sản phẩm	Ở đâu
Lịch và script sao lưu	k8s/backup/cronjob.yaml, k8s/backup/backup.sh
Script khôi phục, chép ra ngoài, thử lỗi, so sánh	scripts/restore.sh, chep-sao-luu.sh, loi-sao-luu.sh, so-sanh-sao-luu.sh
Kiểm thử 3	kt3-dung-dich-vu.sh, kt3-mat-du-lieu.sh, do-gian-doan.sh
Diễn tập sự cố	chaos.sh
Kiểm thử phân quyền, chặn mạng	kiem-quyen.sh, k8s/policy/network-policy.yaml
Luật cảnh báo	observability/rules/hanni-slo.yml
Kết quả	data/raw/gha-09-11/ket-qua-kt3/, data/rpo-rto.xlsx
Ma trận quyền	so-do-va-bang/bang-tai-nguyen-ip-quyen.xlsx, tab "Phân quyền"
README	ho-so-nop/README.md
Báo cáo cá nhân	RIPT1304-20261-01_NhomXX_06_B24DCCC065_BaoCaoCaNhan.docx

Phần 3. Sao lưu và khôi phục từ đầu

3.1 Sao lưu là gì, vì sao "sao lưu xong" chưa đủ

Sao lưu là chép dữ liệu ra một bản riêng để khi hỏng (xoá nhầm, hỏng ổ, bị tấn công) còn lấy lại được. Ba nguyên tắc:

- **Phải ở nơi khác**: để bản sao chung ổ với bản chính thì hỏng ổ là mất cả hai. Nguyên tắc **3-2-1**: 3 bản, trên 2 loại nơi lưu, 1 bản ở nơi khác hẳn.
- **Phải giữ nhiều mốc thời gian**: xoá nhầm hôm qua mà chỉ còn bản sao lưu hôm nay (đã mất dữ liệu) thì vô dụng. Nhóm giữ 14 bản (14 ngày).
- **Phải khôi phục thử**: sao lưu chưa từng khôi phục thì chưa thể tin. Nhóm khôi phục thật, so số dòng và mã băm, đo thời gian.

> Đồng bộ thư mục (như Google Drive) **không phải** sao lưu: xoá nhầm ở máy thì bản trên mây cũng bị xoá theo.

3.2 RPO và RTO (2 con số quan trọng nhất của phần bạn)

- **RPO** (mất tối đa bao nhiêu dữ liệu): tính từ lần sao lưu gần nhất tới lúc hỏng. Sao lưu 2 giờ sáng mỗi ngày, 10 giờ tối hỏng thì mất 20 giờ dữ liệu. RPO của bản sao lưu hằng ngày **tối đa khoảng 1 ngày**.
- **RTO** (mất bao lâu mới dùng lại được): tính từ lúc hỏng tới lúc hệ thống chạy lại đúng. Lab đo được **52 giây** khi xoá hẳn CSDL.

Muốn RPO nhỏ hơn 1 ngày thì cần **nhật ký thay đổi (WAL)**: PostgreSQL ghi mọi thay đổi vào nhật ký, nhóm lưu nhật ký này liên tục (tối đa 60 giây một lần), nên khôi phục được về **đúng thời điểm** trước khi xoá nhầm, chỉ mất tối đa khoảng 1 phút. Phần này Đăng làm sâu; bạn chỉ cần hiểu ý.

3.3 Sao lưu chạy như thế nào trong lab

Lịch chạy: một **CronJob** (việc chạy theo lịch của Kubernetes) tên `hanni-backup`.

hanni-server/infra/k8s/backup/cronjob.yaml, dòng 16–21 (mở trên GitHub):

```
  16 |   # 19:00 UTC là 02:00 giờ Việt Nam. Giờ chạy trên chính cụm nên chính xác, không trễ như
lịch của GitHub Actions.
  17 |   schedule: "0 19 * * *"
  18 |   concurrencyPolicy: Forbid
  19 |   successfulJobsHistoryLimit: 3
  20 |   failedJobsHistoryLimit: 3
  21 |   jobTemplate:
```

Cách đọc lịch cron `"0 19 * * *"`: 5 ô lần lượt là **phút, giờ, ngày trong tháng, tháng, thứ trong tuần**; `*` nghĩa là "mọi". Vậy là "phút 0, giờ 19, mọi ngày". Giờ ở đây là **giờ UTC**, cộng 7 ra giờ Việt Nam: 19 + 7 = 26, tức **2 giờ sáng** hôm sau. `concurrencyPolicy: Forbid`: lần trước chưa xong thì không chạy chồng lần sau.

Script sao lưu backup.sh, từng bước:

hanni-server/infra/k8s/backup/backup.sh, dòng 12–40 (mở trên GitHub):

```
tries=0
until pg_isready --quiet --timeout=2; do
  tries=$((tries + 1))
  [ "$tries" -lt 30 ] || { log "không kết nối được CSDL"; exit 1; }
  sleep 2
done

log "bắt đầu sao lưu CSDL"
pg_dump --format=custom --no-owner --file="$dump.tmp"
pg_restore --list "$dump.tmp" >/dev/null
mv "$dump.tmp" "$dump"

log "bắt đầu sao lưu học liệu"
tar -czf "$media.tmp" -C /srv hoc-lieu
tar -tzf "$media.tmp" >/dev/null
mv "$media.tmp" "$media"

(cd /backup && sha256sum "$(basename "$dump")" "$(basename "$media")" >
"hanni-$stamp.sha256")

keep=${BACKUP_KEEP:-14}
for pattern in 'hanni-*.dump' 'hoc-lieu-*.tar.gz' 'hanni-*.sha256'; do
  # shellcheck disable=SC2086
  ls -1t /backup/$pattern 2>/dev/null | tail -n +"$((keep + 1))" | while read -r old; do
    rm -f -- "$old"
  done
done

log "xong: $(du -h "$dump" | cut -f1) CSDL, $(du -h "$media" | cut -f1) học liệu"
```

1. **Chờ CSDL sẵn sàng** (`pg_isready`, tối đa 30 lần × 2 giây). Không tới được thì báo "không kết nối được CSDL" và thoát mã lỗi.
2. **Xuất CSDL** bằng `pg_dump --format=custom` ra tệp tạm `.tmp`, rồi `pg_restore --list` để **tự kiểm tra** tệp đọc được, xong mới đổi tên thành `.dump`. Ghi ra tệp tạm trước nên bản lỗi không bao giờ đè lên bản tốt.
3. **Nén học liệu** bằng `tar -czf` (cũng qua tệp tạm và tự kiểm tra).
4. **Ghi mã băm SHA-256** của 2 tệp vào tệp `.sha256` để sau này kiểm tra tệp có bị hỏng không.
5. **Xoá bản cũ**, chỉ giữ `BACKUP_KEEP` bản mới nhất (14).

Mọi lỗi đều làm script thoát với mã khác 0 (`set -eu`), nên CronJob hiện **thất bại** thay vì âm thầm giữ tệp hỏng.

Đọc nhật ký một lần sao lưu thật (lượt đo chính thức 09/11, chạy tay bằng `create job --from=cronjob/hanni-backup sao-luu-tay`):

data/raw/gha-09-11/ket-qua-kt3/sao-luu/nhat-ky-sao-luu.txt (mở trên GitHub):

```
2026-11-09T12:10:33Z bắt đầu sao lưu CSDL
2026-11-09T12:10:45Z bắt đầu sao lưu học liệu
2026-11-09T12:10:47Z xong: 81.4M CSDL, 7.4M học liệu
```

Mỗi dòng bắt đầu bằng giờ UTC (chữ `Z` ở cuối là UTC; cộng 7 ra giờ Việt Nam). Dòng 1 → dòng 2: xuất CSDL mất 12 giây. Dòng 2 → dòng 3: nén học liệu mất 2 giây. Dòng "xong" ghi dung lượng: CSDL 81,4 MB, học liệu 7,4 MB. **Thiếu dòng "xong" là lần sao lưu đó hỏng**, xem dòng cuối để biết hỏng ở bước nào (ví dụ "không kết nối được CSDL").

Trạng thái job và lịch, đọc bằng `scripts/kubectl -n hanni get cronjob,job`:

data/raw/gha-09-11/ket-qua-kt3/sao-luu/cronjob.txt (mở trên GitHub):

```
NAME                    STATUS     COMPLETIONS   DURATION   AGE
job.batch/sao-luu-tay   Complete   1/1           24s        24s
NAME                         SCHEDULE     TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
cronjob.batch/hanni-backup   0 19 * * *   <none>     False     0        <none>          23m
```

`COMPLETIONS 1/1` và `STATUS Complete` là thành công (`0/1`, `Failed` là thất bại). `LAST SCHEDULE <none>` vì lab mới dựng 23 phút, chưa tới giờ chạy theo lịch; lần trên là chạy tay. Bản sao lưu được chép ra ngoài cụm rồi kiểm lại mã băm (`sha256sum -c`), `OK` là tệp nguyên vẹn:

data/raw/gha-09-11/ket-qua-kt3/sao-luu/chep-ngoai-cum.txt (mở trên GitHub):

```
hanni-20261109-121031.dump: OK
hoc-lieu-20261109-121031.tar.gz: OK
```

3.4 pg_dump và pg_basebackup (phần mở rộng tự chọn của bạn)

	pg_dump (xuất dữ liệu)	pg_basebackup (chép cả ổ dữ liệu)
Là gì	Xuất nội dung các bảng ra 1 tệp	Chép nguyên thư mục dữ liệu của PostgreSQL
Thời gian sao lưu	12,0 giây	5,0 giây
Dung lượng	82 MB (nén)	556 MB
Thời gian khôi phục	32,7 giây (phải dựng lại chỉ mục)	6,9 giây (chép về là chạy)
Lấy lại 1 bảng	Được	Không
Về đúng thời điểm	Không	Được, kèm nhật ký WAL

Đo trên CSDL có 1 triệu dòng, trung bình 3 lần, số dòng sau khôi phục khớp cả 2 cách. Kết luận: nhóm **dùng cả 2**: bản xuất hằng ngày (nhỏ, chép ra ngoài dễ, lấy lại từng bảng), bản chép cả ổ kèm WAL (khôi phục nhanh, về đúng thời điểm).

3.5 Thử sao lưu khi có lỗi

Script loi-sao-luu.sh cố tình làm hỏng 3 kiểu, kết quả thật:

Lỗi giả lập	Script sao lưu phản ứng	Bản cũ
Đích hết chỗ	Báo "No space left on device", job thất bại	Không bị ảnh hưởng
Đích chỉ đọc	Báo "Read-only file system", job thất bại	Không bị ảnh hưởng
Không tới được CSDL	Báo "không kết nối được CSDL", job thất bại	Không bị ảnh hưởng

> Sao lưu hỏng mà không ai biết còn nguy hiểm hơn không sao lưu. Vì vậy phải thử cả trường hợp lỗi.

Phần 4. Kiểm thử 3 và diễn tập sự cố

4.1 Kiểm thử 3, phần dừng dịch vụ

Script kt3-dung-dich-vu.sh làm 3 lần:

1. Bắt đầu dò API mỗi giây (ghi mã trả về).
2. **Dừng hẳn API** (đưa số bản về 0).
3. Chờ Alertmanager báo cảnh báo, ghi lại sau bao nhiêu giây.
4. **Bật lại** 2 bản, đo bao lâu thì có trả lời.
5. Chạy Kiểm thử 1 để chắc học liệu còn đúng.

Lần	Cảnh báo "từ ngoài không gọi được" kêu sau	Cảnh báo "không còn bản API" kêu sau	Bật lại tới khi có trả lời	Học liệu
1	63 giây	136 giây	5 giây	30/30 tệp đúng
2	84 giây	146 giây	5 giây	30/30 tệp đúng
3	84 giây	146 giây	4 giây	30/30 tệp đúng

Giải thích vì sao cảnh báo kêu sau khoảng 1 phút chứ không ngay lập tức:

hanni-server/infra/observability/rules/hanni-slo.yml, dòng 67–73 (mở trên GitHub):

```
67 |       - alert: HanniEdgeDown
68 |         expr: avg_over_time(probe_success{job="blackbox"}[1m]) < 0.5
69 |         for: 30s
70 |         labels:
71 |           severity: page
72 |         annotations:
73 |           summary: Từ ngoài vào không gọi được API
```

- `expr`: tỉ lệ lần dò thành công **trung bình trong 1 phút** dưới 50%. Dò mỗi vài giây, nên phải hỏng khoảng nửa phút thì trung bình mới xuống dưới 50%.
- `for: 30s`: điều kiện phải đúng liên tục 30 giây nữa mới báo (tránh báo nhầm vì 1 lần chập chờn).
- Cộng lại khoảng 60–90 giây. Đổi được (mục 6.4), nhưng ngắn quá sẽ báo nhầm nhiều.

Thời gian gián đoạn thực tế do người vận hành quyết định (dừng bao lâu thì gián đoạn bấy lâu). Thứ đo được là **phát hiện** (khoảng 1 phút) và **khôi phục** (4–5 giây).

4.2 Kiểm thử 3, phần mất dữ liệu

Script kt3-mat-du-lieu.sh (tự dừng nếu thấy tài khoản không phải dữ liệu giả):

1. CSDL có sẵn 1 triệu dòng nhật ký học, 600 bài thi giả. Chạy sao lưu.
2. **Ghi thêm 1 dòng sau lúc sao lưu** (để chứng minh RPO).
3. **Xoá hẳn CSDL** và **1 tệp học liệu**.
4. Khôi phục từ bản sao lưu.
5. So số dòng 6 bảng chính, kiểm tra dòng ghi thêm ở bước 2, chạy Kiểm thử 1 cho học liệu.

Kết quả: Số dòng 6 bảng chính khớp; dòng ghi sau lúc sao lưu bị mất, đúng như RPO của bản dump; RTO 52 giây từ lúc xoá tới khi kiểm tra xong. Dòng ghi thêm ở bước 2 **bị mất** là **đúng thiết kế**: nó nằm sau lúc sao lưu, chứng minh RPO bằng thời điểm bản sao lưu gần nhất.

4.3 Diễn tập sự cố (chaos)

Script chaos.sh cố tình làm hỏng từng thành phần, đồng thời do-gian-doan.sh gọi API mỗi giây:

Làm hỏng	Gián đoạn dài nhất	Vì sao
Tắt 1 bản API	0 giây	Bản API còn lại vẫn trả lời, Kubernetes tự tạo lại bản đã chết
Tắt Redis	0 giây	Giới hạn request chờ Redis tối đa 150 ms rồi cho qua, API vẫn trả lời
Tắt PostgreSQL	16 giây	Không có CSDL thì API báo chưa sẵn sàng; PostgreSQL tự bật lại thì API tự phục hồi
Tắt 1 node	94 giây	Kubernetes cần thời gian nhận ra node mất rồi mới dời pod sang node còn lại

Cách nói khi bảo vệ: "Mất 1 bản API hay mất Redis thì người dùng không thấy gì. Mất CSDL thì gián đoạn khoảng 16 giây. Mất cả 1 node thì khoảng 1,5 phút, vì lab chỉ có 2 node và Kubernetes cần thời gian nhận ra node đã mất."

4.4 Runbook: khi xảy ra sự cố thì làm gì

Sự cố	Phát hiện bằng	Cách xử lý	Mục tiêu
API ngừng trả lời	Cảnh báo gọi từ ngoài không được	`scripts/kubectl -n hanni get pods`, xem `logs`, bật lại hoặc tăng số bản	Có trả lời lại sau vài giây
Mất dữ liệu CSDL	Lỗi khi đọc ghi, số dòng bất thường	`scripts/restore.sh`: khôi phục bản mới nhất vào CSDL sạch, so số dòng	Mất tối đa 1 ngày (bản dump) hoặc 1 phút (có WAL)
Mất 1 tệp học liệu	Kiểm thử 1 báo tệp sai mã băm	Lấy lại từ gói học liệu đã sao lưu, so mã băm	Học liệu đúng 100%
Đĩa sao lưu đầy	Job sao lưu báo thất bại	Dọn bản cũ hoặc thêm đĩa, chạy lại job	Không để bản lỗi đè bản tốt

Phần 5. Phân quyền và chặn mạng

5.1 Ma trận quyền

Tài nguyên	Khách	Người học	Giáo viên	Quản trị
Danh sách học liệu	Xem	Xem	Xem	Xem
Video bài giảng	Được mời đăng nhập	Xem	Xem	Xem
PDF, bài nghe MP3	Tải	Tải	Tải	Tải
Tiến độ học	Từ chối	Chỉ của mình	Học viên lớp mình	Theo nhu cầu
Trang số liệu nội bộ `/api/metrics`	Không thấy (404)	Không thấy	Không thấy	Chỉ trong cụm
CSDL, Redis	Không tới được	Không tới được	Không tới được	Chỉ trong cụm

5.2 Phân biệt 401, 403, 404, 410 (hay bị hỏi)

- **401**: "bạn là ai?" chưa đăng nhập. Khách xin video cần đăng nhập.
- **403**: "biết bạn là ai nhưng không cho". Người học gọi chức năng quản trị; hoặc liên kết bị sửa chữ ký.
- **404**: "không có thứ này". Dùng để **giấu** `/api/metrics` khỏi người ngoài: nói "không có" an toàn hơn nói "có nhưng cấm".
- **410**: "từng có nhưng hết hạn". Liên kết học liệu quá 10 phút.

5.3 Kiểm thử phân quyền: 9 ca

Script kiem-quyen.sh dùng **tài khoản giả, không dùng quyền quản trị** (quản trị có toàn quyền nên luôn thành công, không chứng minh được gì). Kết quả thật (lượt 09/11):

```
Q1   Khách xin liên kết R1 cần đăng nhập                  mong đợi 401        thực tế 401        ĐẠT
Q2   Dùng liên kết đã hết hạn                             mong đợi 410        thực tế 410        ĐẠT
Q3   Sửa chữ ký trên liên kết                              mong đợi 403        thực tế 403
ĐẠT
Q4   Người học xin liên kết R1 đúng quyền                mong đợi 200        thực tế 200        ĐẠT
Q5   Người học xem tiến độ lớp của người khác     mong đợi 403|404    thực tế 403        ĐẠT
Q6   Máy ngoài cụm nối thẳng Postgres 10.42.1.9:5432        mong đợi dong       thực tế dong
ĐẠT
Q6   Máy ngoài cụm nối Redis qua 127.0.0.1:6379               mong đợi dong       thực tế dong
ĐẠT
Q7   Pod API nối PgBouncer, đường được phép              mong đợi ket-noi    thực tế ket-noi    ĐẠT
Q7   Pod API nối thẳng Postgres, bỏ qua PgBouncer             mong đợi chan       thực tế chan
ĐẠT
Q8   Từ ngoài cụm gọi /api/metrics                          mong đợi 404        thực tế 404
ĐẠT
Q9   Người học gọi route chỉ dành cho quản trị        mong đợi 403        thực tế 403        ĐẠT
```

5.4 Chặn mạng trong cụm (NetworkPolicy)

hanni-server/infra/k8s/policy/network-policy.yaml, dòng 1–13 (mở trên GitHub):

```
  1 | # Mặc định chặn hết chiều vào, chỉ mở đúng đường đi của kiến trúc: ingress → API/media, API →
PgBouncer/Redis,
  2 | # PgBouncer → Postgres. Prometheus vào được cổng số đo. API lỡ bị chiếm cũng không chạm thẳng
được Postgres.
  3 | apiVersion: networking.k8s.io/v1
  4 | kind: NetworkPolicy
  5 | metadata:
  6 |   name: default-deny-ingress
  7 | spec:
  8 |   podSelector: {}
  9 |   policyTypes: [Ingress]
 10 | ---
 11 | apiVersion: networking.k8s.io/v1
 12 | kind: NetworkPolicy
 13 | metadata:
```

Ý tưởng: **mặc định chặn mọi kết nối vào**, rồi chỉ mở đúng các đường cần: cổng vào tới API và Nginx; API, worker tới Redis và PgBouncer; **chỉ PgBouncer tới PostgreSQL**; Prometheus lấy số liệu. Nhờ vậy, nếu API bị chiếm quyền cũng không nối thẳng được vào PostgreSQL (ca Q7 kiểm tra đúng điều này).

Phần 6. Sửa và cấu hình trực tiếp

Bảng đầy đủ cho cả nhóm ở infra/README.md, mục "Sửa nhanh khi bảo vệ".

6.1 Chạy sao lưu ngay và khôi phục ngay

```
scripts/kubectl -n hanni create job --from=cronjob/hanni-backup sao-luu-tay   # sao lưu ngay, không
chờ 2 giờ sáng
scripts/kubectl -n hanni logs job/sao-luu-tay                                  # xem nhật ký: "xong:
... CSDL, ... học liệu"
scripts/restore.sh                                                             # khôi phục bản mới
nhất vào CSDL sạch, so số dòng
```

`restore.sh` khôi phục vào một CSDL **riêng** tên `hanni_khoi_phuc`, không đụng CSDL đang phục vụ, rồi in số dòng 2 bên để so. Muốn chạy lại job sao lưu tay lần nữa thì đổi tên (`sao-luu-tay-2`).

6.2 Đổi giờ sao lưu

hanni-server/infra/k8s/backup/cronjob.yaml, dòng 16–18 (mở trên GitHub):

```
  16 │   # 19:00 UTC là 02:00 giờ Việt Nam. Giờ chạy trên chính cụm nên chính xác, không trễ như
lịch của GitHub Actions.
  17 │   schedule: "0 19 * * *"
  18 │   concurrencyPolicy: Forbid
```

Ví dụ muốn 23 giờ tối giờ Việt Nam: 23 − 7 = 16 giờ UTC → `"0 16 * * *"`. Chạy `make deploy`, kiểm tra `scripts/kubectl -n hanni get cronjob` (cột SCHEDULE). Mỗi 6 giờ một lần: `"0 */6 * * *"` (RPO giảm từ 1 ngày còn 6 giờ, đổi lại tốn dung lượng hơn).

Thêm ví dụ để tập (ô thứ 5 là thứ trong tuần: 0 = Chủ nhật, 1 = thứ Hai … 6 = thứ Bảy; `*/6` là "cứ 6", `1-5` là "từ 1 tới 5", `1,15` là "1 và 15"):

Muốn (giờ Việt Nam)	Viết (giờ UTC)	Cách nghĩ
02:00 mỗi ngày (hiện tại)	`0 19 * * *`	2 − 7 = −5, cộng 24 = 19 giờ **hôm trước**
23:00 mỗi ngày	`0 16 * * *`	23 − 7 = 16, cùng ngày
12:30 trưa mỗi ngày	`30 5 * * *`	Phút 30, giờ 12 − 7 = 5
Mỗi 30 phút	`*/30 * * * *`	Không phụ thuộc múi giờ
Mỗi 6 giờ	`0 */6 * * *`	Giờ 0, 6, 12, 18 UTC = 7, 13, 19, 1 giờ Việt Nam
02:00 từ thứ Hai tới thứ Sáu	`0 19 * * 0-4`	**Bẫy**: 02:00 thứ Hai ở Việt Nam là 19:00 **Chủ nhật** UTC, nên thứ lùi 1 ngày: 0–4, không phải 1–5
03:00 Chủ nhật hằng tuần	`0 20 * * 6`	03:00 Chủ nhật VN = 20:00 thứ Bảy UTC

Để khỏi phải đổi giờ, CronJob (Kubernetes từ bản 1.27, lab đang dùng 1.31) cho thêm dòng `timeZone: "Asia/Ho_Chi_Minh"` ngay dưới `schedule:`, khi đó viết thẳng giờ Việt Nam (`"0 2 * * *"`). Nhóm giữ giờ UTC vì đó là mặc định và nhật ký cũng ghi giờ UTC, so cho dễ.

6.3 Đổi số bản giữ lại

hanni-server/infra/k8s/backup/cronjob.yaml, dòng 46–47 (mở trên GitHub):

```
46 |                 - name: BACKUP_KEEP
47 |                   value: "14"
```

Đổi `"14"` thành ví dụ `"7"`, `make deploy`. Chạy sao lưu tay vài lần rồi đếm tệp trong thư mục sao lưu: chỉ còn 7 bộ mới nhất.

6.4 Đổi ngưỡng cảnh báo

hanni-server/infra/observability/rules/hanni-slo.yml, dòng 67–69 (mở trên GitHub):

```
67 |       - alert: HanniEdgeDown
68 |         expr: avg_over_time(probe_success{job="blackbox"}[1m]) < 0.5
69 |         for: 30s
```

Ví dụ muốn báo nhanh hơn: đổi `for: 30s` thành `for: 10s`. Chạy `make observability`, rồi `scripts/kubectl -n observability rollout restart deployment/prometheus`. Kiểm tra: dừng API (`scripts/kubectl -n hanni scale deployment hanni-api --replicas=0`), mở http://alertmanager.hanni.localhost:18080, đếm thời gian tới khi cảnh báo hiện; nhớ bật lại (`--replicas=2`).

6.5 Diễn tập 1 sự cố trước mặt thầy

Mở 2 cửa sổ terminal trong `hanni-server/infra`:

```
# cửa sổ 1: dò mỗi giây, Ctrl+C để in thời gian gián đoạn dài nhất
scripts/do-gian-doan.sh k6/ket-qua/dien-tap.csv
# cửa sổ 2: tắt đột ngột 1 bản API
scripts/chaos.sh api
```

Chờ khoảng 30 giây, Ctrl+C ở cửa sổ 1: thấy gián đoạn 0 giây. Mở Grafana thấy Kubernetes tạo lại bản mới.

6.6 Đổi quyền

- Học liệu cần hay không cần đăng nhập: `requiresLogin` trong `k8s/media-manifest.json` (phần của Thành), sau đó `make deploy` + khởi động lại API.
- Ai được nối tới CSDL, Redis: k8s/policy/network-policy.yaml, `make deploy`, chạy lại `scripts/kiem-quyen.sh` để thấy ca kết nối thẳng CSDL vẫn bị chặn.

Phần 7. Đánh giá mở rộng, chạy lại và số liệu

7.1 Đánh giá mở rộng theo số đo

Nguyên tắc: **chỉ thêm khi số đo chạm ngưỡng**, không làm trước. Bảng tính tài nguyên cho từng quy mô:

Chỉ tiêu	100	1.000	10.000	100.000
Tự học cùng lúc	7	63	625	6.250
Xem video cùng giờ	60	300	1.500	5.000
Băng thông video, có dự phòng (Mbps)	178	888	4.441	14.802
Số bản API ước lượng	2	2	5	50

Quy mô	Việc phải làm	Chưa cần	Chuyển mức khi
100	Giới hạn request theo tài khoản; sao lưu đã thử khôi phục	Kubernetes, bản sao, CDN	Máy hết RAM liên tục hoặc p95 đọc vượt 300 ms
1.000	Nhiều bản API tự thêm theo CPU, worker riêng, PgBouncer, video qua Nginx/CDN, giám sát đủ	Chia nhỏ CSDL	Đã đủ 6 bản mà CPU vẫn trên 70%
10.000	CSDL máy riêng có bản sao, kho tệp + CDN bắt buộc, hàng đợi việc, chia bảng lớn theo tháng	Chia dữ liệu theo người dùng	CPU CSDL chính trên 70% lúc cao điểm
100.000	Nhiều vùng, tự chuyển CSDL khi hỏng bằng công cụ có sẵn, tách kho phân tích	Tự viết cơ chế đồng thuận	Chi phí và số người vận hành

Lý do dựa trên điểm nghẽn đã đo: ở 1.000, điểm nghẽn đầu tiên là hàng chờ kết nối CSDL, sửa xong chuyển sang CPU của API (thêm bản là xong); lên 10.000 thì CSDL thành điểm nghẽn, phải tách máy riêng và có bản sao; video thì luôn phải qua CDN.

7.2 Chạy lại phần của bạn, từng bước

1. Dựng lab (mục 1.6).
2. Sao lưu tay + khôi phục (mục 6.1): số dòng 2 bên khớp.
3. Kiểm thử 3: `scripts/kt3-dung-dich-vu.sh k6/ket-qua/kt3 3` (khoảng 10 phút), rồi `scripts/kt3-mat-du-lieu.sh`.
4. Phân quyền: `scripts/kiem-quyen.sh k6/ket-qua/quyen.txt` → mọi dòng ĐẠT.
5. Diễn tập: `scripts/chaos.sh api` cùng `do-gian-doan.sh` (mục 6.5).
6. Thử lỗi sao lưu: `scripts/loi-sao-luu.sh k6/ket-qua/loi-sao-luu.txt`.

7.3 Số liệu để nói khi bảo vệ

Câu cần nói	Số
Dừng API: cảnh báo kêu sau	63, 84, 84 giây
Bật lại: có trả lời sau	5, 5, 4 giây; học liệu 30/30 đúng cả 3 lần

Câu cần nói	Số
Mất dữ liệu	Số dòng khớp, RTO 52 giây, dòng ghi sau sao lưu mất (đúng RPO)
Tắt từng thành phần	mất 1 bản API 0 s, mất Redis 0 s, mất Postgres 16 s, mất 1 node 94 s
Sao lưu pg_dump / pg_basebackup	82 MB, khôi phục 33 giây / 556 MB, 7 giây
Thử lỗi sao lưu	3/3 trường hợp báo lỗi đúng, bản cũ không bị ảnh hưởng
Phân quyền	11/11 dòng đạt

7.4 README của hồ sơ: viết gì, vì sao, kiểm thế nào

Phiếu đề yêu cầu README ghi **phiên bản, cách dựng lại, cách chạy kiểm thử**. Bạn phụ trách ho-so-nop/README.md và infra/README.md. Mỗi mục trong README hồ sơ trả lời 1 câu người chấm sẽ hỏi:

Mục	Trả lời câu	Khi nào phải sửa
Đầu tệp, bảng thành viên	Lớp, nhóm, ai làm phần nào, mã sinh viên	Khi biết số nhóm
Nội dung hồ sơ	Mỗi thư mục, tệp nộp chứa gì	Thêm hoặc bớt tệp nộp
Mã nguồn đầy đủ	Repo, nhánh, **commit** nào đúng với hồ sơ	Mỗi lần đẩy commit mới lên nhánh
Phiên bản	Docker, k3s, Node.js, PostgreSQL, k6... bản nào	Đổi image hoặc công cụ
Yêu cầu máy	Máy thế nào thì dựng được	Lab nặng hơn
Dựng lại hệ thống	6 lệnh `make`, thời gian dựng, cách chạy web	Đổi Makefile
Chạy kiểm thử	Mỗi ca: lệnh, kết quả nằm ở đâu, tiêu chí đạt	Thêm ca kiểm thử
Sửa cấu hình khi bảo vệ	Trỏ tới bảng "Sửa nhanh" trong infra/README.md	Thêm thông số hay bị hỏi
Nguồn tham khảo, dùng AI	Nguồn dữ liệu, giấy phép, nhóm dùng trợ lý AI thế nào	Thêm nguồn

Kiểm README đúng hay chưa không phải bằng đọc lại, mà bằng **làm theo trên máy sạch**: mỗi lượt đo chính thức, máy ảo GitHub mới tinh dựng lab chỉ bằng đúng các lệnh trong README (nhật ký ở evidence/dung-lai/ và `data/raw/gha-09-11/ket-qua-*/dung-lab.txt`). Lệnh nào trong README sai thì lượt đó hỏng ngay. Muốn tự thử: clone repo về thư mục mới, làm đúng từng dòng README, ghi lại chỗ vướng. Quy tắc viết: mỗi lệnh chép-dán chạy được; ghi kết quả mong đợi ("mọi pod Running"); không ghi mật khẩu thật (mật khẩu lab sinh ngẫu nhiên lúc `make up`).

Phần 8. Câu hỏi vấn đáp

Trả lời ngắn, có số, rồi mở minh chứng ở cột cuối. Thầy hỏi khác đi một chút thì vẫn dựa vào ý chính của câu trả lời. Cùng nội dung có ở tab **Ôn vấn đáp** của sheet phân công.

Câu hỏi về phần của bạn

RPO khác RTO thế nào?

RPO là lượng dữ liệu tối đa có thể mất (tính từ bản sao lưu gần nhất); RTO là thời gian từ lúc hỏng tới lúc dùng lại được. Sao lưu 1 lần/ngày thì RPO tối đa 1 ngày dù khôi phục nhanh đến đâu.

Mở minh chứng: data/rpo-rto.xlsx

Vì sao bản sao lưu phải để nơi khác?

Hỏng máy hoặc ổ đĩa sẽ mất cả bản chính lẫn bản sao nếu để chung. Nhóm chép ra ngoài cụm và so mã băm; giới hạn là trong lab vẫn cùng 1 máy.

Mở minh chứng: data/raw/gha-09-11/ket-qua-kt3/sao-luu/

Dừng API thì có mất dữ liệu không?

Dữ liệu đã ghi nằm trong PostgreSQL nên không mất; request đang xử lý có thể lỗi. 3 lần dừng: cảnh báo kêu sau 63, 84, 84 giây, bật lại có trả lời sau 5, 5, 4 giây, học liệu đúng 30/30.

Mở minh chứng: data/raw/gha-09-11/ket-qua-kt3/kt3-a/

Vì sao kiểm tra quyền không dùng tài khoản quản trị?

Quản trị có toàn quyền nên luôn thành công, không chứng minh được gì. Phải dùng đúng vai trò: khách, người học.

Mở minh chứng: data/raw/gha-09-11/ket-qua-kt3/quyen.txt

Sao lưu xuất dữ liệu khác chép cả ổ thế nào?

pg_dump: 82 MB, khôi phục 33 giây, lấy lại được từng bảng. pg_basebackup: 556 MB, khôi phục 7 giây, kết hợp nhật ký thay đổi để về đúng thời điểm. Nhóm dùng cả 2.

Mở minh chứng: Báo cáo nhóm mục 5.7

Thao tác có thể bị yêu cầu

Chạy sao lưu tay rồi restore.sh; đổi giờ sao lưu trong k8s/backup/cronjob.yaml (giờ UTC); đổi số bản giữ BACKUP_KEEP; tắt 1 bản API bằng scripts/chaos.sh api và đọc thời gian gián đoạn.

Mở minh chứng: cau-hinh-ma-nguon/hanni-server/infra/README.md, mục "Sửa nhanh khi bảo vệ"

Câu hỏi chung ai cũng phải trả lời được

Vì sao thử 20 kết nối không chứng minh phục vụ được 1.000 sinh viên?

Thời gian trả lời không tăng đều: từ 10 tới 40 lượt/giây p95 chỉ từ 23 lên 52 ms, thêm 10 lượt nữa đã 261 ms. Mức nhỏ chưa chạm giới hạn nào nên không nói gì về điểm gãy. Kết quả chỉ đúng cho đúng máy, dữ liệu, cấu hình đã đo.

Mở minh chứng: Báo cáo nhóm mục 5.3 (đo thăm dò), 6.2

Tài khoản khác người dùng cùng lúc thế nào?

Tài khoản là số người có thể dùng; người dùng cùng lúc mới tạo tải. 1.000 tài khoản sinh ra 63 người khi tự học, 300 khi cả lớp xem video, 600 khi thi thử đồng loạt.

Mở minh chứng: Báo cáo nhóm mục 2.1–2.2; so-do-va-bang/bang-tai-nguyen-ip-quyen.xlsx

Vì sao video không đi qua API?

Cả lớp xem video cần khoảng 888 Mbps trong khi API chỉ khoảng 56 request/giây. API chỉ kiểm quyền và ký liên kết, tệp do Nginx (thực tế là CDN) phát.

Mở minh chứng: Báo cáo nhóm mục 3.2; cau-hinh-ma-nguon/hanni-server/infra/k8s/app/media.yaml

Thêm máy chủ có hết quá tải không?

Không, nếu điểm nghẽn ở CSDL hoặc băng thông. Phải đo để biết điểm nghẽn trước: ở lab, nghẽn đầu tiên là hàng chờ kết nối CSDL, sửa xong mới tới CPU của API.

Mở minh chứng: Báo cáo nhóm mục 6.1; bảng Grafana "Mở rộng"

Bản sao lưu thế nào mới có giá trị?

Khi đã khôi phục thử, so số dòng và mã băm, đo được thời gian. Nhóm đã xoá hẳn CSDL rồi khôi phục: số dòng khớp, RTO 52 giây.

Mở minh chứng: Báo cáo nhóm mục 5.4; data/rpo-rto.xlsx

Lên 10.000 hay 100.000 sinh viên thì làm gì?

10.000: CSDL máy riêng có bản sao, kho tệp + CDN bắt buộc, khoảng 5 bản API. 100.000: nhiều vùng, tự chuyển CSDL khi hỏng, tách kho phân tích. Chuyển mức khi số đo chạm ngưỡng, không làm trước.

Mở minh chứng: Báo cáo nhóm mục 6.4

Phần 9. Việc cần làm trước khi nộp và bảo vệ

Trước khi nộp

Báo cáo cá nhân của bạn `ho-so-nop/bao-cao/ca-nhan/RIPT1304-20261-01_NhomXX_06_B24DCCC065_BaoCaoCaNhan.docx` **đã điền đủ** theo mẫu: mã sinh viên, lớp, ngày, họ tên ở mục 10. Chỉ còn số nhóm (chỗ `XX` trong tên tệp và dòng "Nhóm"), Đăng điền khi thầy công bố. Việc của bạn:

1. Đọc báo cáo nhóm `ho-so-nop/bao-cao/Bao-cao-nhom.pdf` và báo cáo cá nhân của bạn.
2. Đọc kỹ các câu "Tự làm: ..." trong bảng **mục 7 (Tiến độ và phối hợp)** và câu "Tôi tự sửa: ..." ở **mục 9**: đó là những việc báo cáo khẳng định **chính bạn đã làm**. Làm thật đủ các việc đó trước ngày 23/11 (theo mục "Chạy lại" của tài liệu này). Việc nào bạn không làm thì sửa câu đó cho đúng sự thật trước khi nộp.
3. Mục 10 ghi họ tên bạn thay cho chữ ký. **Chỉ để nguyên khi bạn đã tự chạy lại và giải thích được.** Mục 9 đã ghi rõ nhóm dùng trợ lý AI soạn bản nháp; người ký chịu trách nhiệm giải thích.
4. Báo Đăng khi xong (hoặc gửi bản đã sửa) để gom hồ sơ nộp lên Drive của thầy.

Trình bày 3 phút

Lúc bảo vệ, mỗi người có vài phút trình bày phần mình. Khung dưới đây đi đúng thứ tự thầy chấm: việc gì, bằng chứng ở đâu, số đo bao nhiêu, giới hạn là gì. Tập nói to 2–3 lần, bấm giờ.

Thời gian	Nói gì	Mở gì
0:00–0:40	Sao lưu hằng ngày 02:00: pg_dump CSDL + nén học liệu, tự kiểm tra tệp, ghi mã băm, giữ 14 bản, chép ra ngoài cụm; lỗi thì job báo thất bại (3/3 tình huống lỗi)	k8s/backup/cronjob.yaml, nhật ký sao lưu
0:40–1:30	Kiểm thử 3 dừng dịch vụ: dừng hẳn API 3 lần, cảnh báo từ ngoài sau 63, 84, 84 giây, bật lại có trả lời sau 5, 5, 4 giây, học liệu đúng	ket-qua-kt3/kt3-a/
1:30–2:10	Kiểm thử 3 mất dữ liệu: xoá CSDL và 1 tệp học liệu, khôi phục, số dòng khớp, RTO 52 giây; RPO tối đa 1 ngày với bản dump (dòng ghi sau lúc sao lưu mất đúng như dự kiến)	data/rpo-rto.xlsx
2:10–2:40	Diễn tập: mất 1 bản API 0 s, mất Redis 0 s, mất Postgres 16 s, mất 1 node 94 s; phân quyền 11/11 dòng đạt	ket-qua-kt3/chaos/, quyen.txt
2:40–3:00	Mở rộng theo điểm nghẽn: 1.000 → thêm bản API + CDN; 10.000 → CSDL riêng có bản sao; giới hạn: lab 1 máy	Mục 7.1

Trước buổi bảo vệ

1. Tập **mở đúng minh chứng trong 10 giây**: biết tệp kết quả của mình nằm ở đâu trong `ho-so-nop/` (bảng "Sản phẩm của bạn" ở phần 2).
2. Tập **1–2 thao tác** ở phần "Sửa và cấu hình trực tiếp": nói được tệp nào, dòng nào, lệnh áp dụng, cách kiểm tra.
3. Tập trả lời các câu ở phần Câu hỏi vấn đáp bằng lời của mình, có số.

4. Xem lại phần 1 (kiến thức nền): thầy có thể hỏi kiến trúc chung, không chỉ phần của bạn.

> Câu chốt khi bị hỏi khó: "Theo số đo của nhóm trên lab, trong điều kiện … thì … ; giới hạn là … ; muốn chắc hơn thì đo thêm …". Trả lời theo số đo và nêu giới hạn luôn được đánh giá cao hơn khẳng định chung chung.